

AERO: ENTROPY-GUIDED FRAMEWORK FOR PRIVATE LLM INFERENCE

Nandan Kumar Jha¹ Brandon Reagen¹

ABSTRACT

Privacy-preserving computation enables language model inference directly on encrypted data yet suffers from prohibitive latency and communication overheads, primarily due to nonlinear functions. Removing nonlinearities, however, can trigger one of two failure modes restricting the potential for nonlinearity removal: entropy collapse in deeper layers, which destabilizes training, and entropic overload in early layers, causing under-utilization of attention heads. To address these challenges, we introduce AERO, an entropy-guided framework to strategically eliminates costly nonlinear operations from transformer architectures, which employs an adaptive recalibration through a head-wise entropy regularizer with learnable per-head strengths, enabling each head to adjust its entropy level while penalizing extreme entropies and fostering functional diversity through a tolerance margin. Experiments show AERO can save $3.4\times$ communication and $1.4\times$ latency, without any performance penalty.

1 INTRODUCTION

The popularity of large language model (LLMs) has raised privacy concerns as more sensitive user data (e.g., prompts) are sent to the cloud (Staab et al., 2024; Mireshghallah et al., 2024; Priyanshu et al., 2023; Lauren & Knight, 2023). Private inference (PI) leverages cryptography to allow users to interact with service providers’ models without revealing inputs, ensuring data privacy and model weight protection (Carlini et al., 2024). However, deploying private transformer-based LLMs is challenging due to prohibitive communication and latency overheads. For instance, BOLT (Pang et al., 2024) requires 59.6 GB for a single private inference on BERT-base model, while Bumblebee (Lu et al., 2025), despite leveraging homomorphic encryption (HE) for nonlinear operations, still requires 15.5 GB for BERT-large.

Moreover, our experimental study, performed using Bumblebee private LLM inference framework, showed that generating one output token in a private GPT-2 (125M) inference takes 8.2 minutes and 25.3 GBs of communication (Table 5) with 128 input tokens, and at 512 inputs 30.7 minutes and 145.2 GBs of communication (Table 7).

Recent work (Lu et al., 2025; Hou et al., 2023) attribute these overheads primarily to the expensive *nonlinear* operations (e.g., GELU and LayerNorm). While these operations are crucial for model stability and performance, they rely on expensive cryptographic primitives with require numerous rounds of interaction, leading to disproportionate latency and communication costs. For instance, a *single* GELU

^{*}Equal contribution ¹New York University (NYU). Correspondence to: Nandan Kumar Jha <nj2049@nyu.edu>.

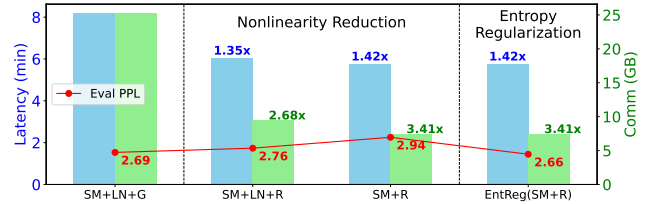


Figure 1. Latency and communication savings through nonlinearity reduction, and performance improvement through entropy regularization when AERO is applied on GPT-2 (125M), trained from scratch on CodeParrot dataset (detailed breakdown in Table 5).

operation in BERT-base model requires 3.9×10^6 operations, each involving multiple secure multiplications and communication rounds, adding 1-2 KB per operation (Lu et al., 2025). Thus, GELU and LayerNorm contribute 59% of communication and 49% of latency in private GPT-2 inference with 256 input tokens (Hou et al., 2023).

Designing LLMs with fewer nonlinear operations offers a promising direction for enabling cryptographically secure private LLM inference. However, this requires a deeper understanding of the fundamental role that nonlinearities in shaping learning dynamics and internal representations—which remains largely unexplored. The state-of-the-art private LLM systems optimize for how efficiently inference is performed under cryptography settings *rather* than which nonlinearity the model requires. Specifically, THOR (Moon et al., 2025) accelerates HE execution using diagonal-major packing and tailored the protocols for nonlinear operations, NEXUS (Zhang et al., 2025a) makes PI non-interactive under CKKS FHE, and BLB (Xu et al., 2025) reduces the number of truncations and conversion in a hybrid system by fine-grained operator splitting and fusion.

Nonetheless, some recent studies have examined the LLM

failure modes such as training instability (Wortsman et al., 2024; Rybakov et al., 2024) and rank collapse (Bao et al., 2024; Noci et al., 2022). However, the critical questions remain, specifically: (i) How does removing LLM nonlinearities affect learning dynamics and information flow? (ii) Do we need all types of nonlinearity or can we use only those vital, e.g., Softmax for self-attention?

To address this, we introduce an information-theoretic framework to systematically analyze the role of nonlinearities in transformer-based LLMs. Using Shannon’s entropy as a quantitative lens, we uncover the **dual** significance of nonlinearities: (i) they preserve the representational diversity in attention heads, particularly in early layers, fostering head-wise specialization; and (ii) maintain stable training by preventing entropy collapse in deeper layers.

We observed a new phenomenon, *entropic overload*, where many attention heads remain stuck in greater entropy states during training when *crucial* nonlinearities are removed. Our empirical results suggest that entropic overload disrupts information flow, impairing the model’s ability to learn diverse representations and increases perplexity (Table 2).

To mitigate entropic overload, we propose an entropy-guided attention mechanism paired with a novel entropy regularization scheme that strategically penalizes the deviation from well-behaved entropy distribution through two key innovations: (i) *Headwise learnable thresholds* to dynamically adjust regularization strength for each attention head; (ii) *Tolerance margins* to prevent over-regularization by allowing only a small fraction of attention heads to attain higher entropy values. These assure diversity while controlling excessive penalization in the absence of nonlinearities.

Furthermore, we establish the effectiveness of PI-friendly static alternatives to LayerNorm, such as weight and spectral normalization (Salimans & Kingma, 2016; Miyato et al., 2018), which do not require nonlinear operation at inference. Unlike stabilization methods that heavily rely on LayerNorm layers, such as QK-LayerNorms (Dehghani et al., 2023; Wortsman et al., 2024; Muennighoff et al., 2024) or the positional variants like MixLN (Li et al., 2025), static normalization methods maintain training stability by preventing entropy collapse in deeper layers, and avoid the cost of inverse square-root operation at inference.

To this end, we summarize our contributions as follows.

- **Conceptual Insights:** Using Shannon’s entropy as a quantitative lens, we analyze how nonlinearities regulate internal information flow and stabilize training in transformer-based LLMs, offering design insights for tailoring existing LLM architecture for efficient PI (§4).
- **Framework:** We propose AERO, a three-stage design framework that systematically reduces nonlinearities in LLMs while maintaining training stability, showing that models remain trainable even with Softmax as the only

nonlinearity (§5).

- **Architectural:** We explore static alternatives to LayerNorm which effectively prevent entropy collapse and maintain training stability without incurring the overheads of nonlinear operations at inference (§5.1).
- **Algorithmic:** We introduce an entropy-guided attention mechanism paired with a novel entropy regularization scheme which prevents entropic overload in the absence of nonlinearities. We use entropy as an actionable cause and penalize extreme entropy values while adapting the regularization strength to the head-specific role (§5.2).
- **Empirical:** We demonstrate the scalability and efficacy of AERO across context sizes (128, 256, 512), model depths (12 and 18 layers), and training scales (1.2B to 4.8B tokens) on the CodeParrot and Languini book datasets (§6). AERO achieves $3.4\times$ communication and $1.42\times$ latency reduction with no loss in performance (Figure 1). Even under the most-stringent stringent nonlinearity setting, where Softmax is the only remaining nonlinearity, entropy regularization improves perplexity by $6\%–8\%$. We also conducted extensive benchmarking against state-of-the-art method (He & Hofmann, 2024).

2 RELATED WORK

Role of nonlinearity in LLMs Prior work used a simplified one-layer transformer architecture—a single Softmax-based self-attention head and a ReLU-based feed-forward network (FFN)—to investigate the role of attention and FFN nonlinearity for in-context learning tasks (Li et al., 2024). Other explores a broader range of nonlinear architectures and in-context learning tasks, showing that the optimal activation function can vary depending on the specific function class the model is attempting to learn (Cheng et al., 2024).

To the best of our knowledge, the most closely related work is by Brody et al. (2023), which showed the LayerNorm’s geometric regularization effect on attention expressivity. However, their analysis is specific to LayerNorm, while our study takes a more holistic approach—quantifying how each nonlinearity affects the entropy dynamics of attention mechanism using a unified ablation framework.

Simplifying nonlinearity in transformer models Existing approaches to simplifying nonlinearities in transformer models generally follow two main strategies. The first centers on architectural optimizations, particularly the design of normalization-free models. While these models remove LayerNorms, they retain nonlinearities such as GELU (He & Hofmann, 2024; Noci et al., 2023; He et al., 2023), or even replace LayerNorm with $\tanh$ (Zhu et al., 2025)—both of which remain expensive to compute under encryption. Consequently, their efficiency in plaintext settings often fails to translate effectively to cryptographic settings.

The second approach approximates nonlinear functions with polynomials, mainly targeting GELU and inverse-square-root in LayerNorms (Zimerman et al., 2024; Dhyani et al., 2024). While promising, these approximations come with their own set of challenges: they tend to work only within narrow input ranges and can make training unstable, particularly when high-degree polynomials are involved (Knott et al., 2021; Zimerman et al., 2024). Additionally, a few prior on private transformer-based models overlook the cost of LayerNorm (Li et al., 2023a; Zeng et al., 2023; Zhang et al., 2023; Chen et al., 2023).

These findings, while valuable, do not address the comprehensive role of nonlinearities in maintaining model stability and fostering attention head diversity in a multi-layer LLM, limiting their utility for private LLM design. We address this gap with an information-theoretic framework that analyzes how nonlinearities shape learning dynamics at *pre-training*. Refer to Appendix A for additional related work.

3 BACKGROUND

We first give an overview of the Transformer architecture in decoder-only LLMs, followed by the threat model and cryptographic protocols details. We also discuss our methodological considerations for characterizing nonlinearities.

3.1 Transformer Architecture in Decoder-only LLMs

Notations We denote the number of layers as L , number of heads as H , model dimensionality as d , head dimension as d_k (where $d_k = \frac{d}{H}$), and context length as T .

An overview of transformer-based decoder-only architecture. A transformer-based LLM is constructed by sequentially stacking L transformer blocks, where each block is composed of two sub-blocks: an attention mechanism and a feed-forward network (FFN), both having their own residual connections and normalization layers (PreLN (Xiong et al., 2020)). Formally, transformer blocks take an input sequence $\mathbf{X}_{\text{in}} \in \mathbb{R}^{T \times d}$, consisting of T tokens of dimension d , and transform it into $\mathbf{X}_{\text{out}}$ as follows:

$$\mathbf{X}_{\text{out}} = \hat{\mathbf{X}}_{\text{SA}} + \text{FFN}_{\text{GELU}}(\text{LayerNorm}_2(\hat{\mathbf{X}}_{\text{SA}})), \quad (1)$$

where $\hat{\mathbf{X}}_{\text{SA}} = \mathbf{X}_{\text{in}} + \text{MHA}(\text{LayerNorm}_1(\mathbf{X}_{\text{in}}))$

The Multi-Head Attention (MHA) sub-block enables input contextualization by sharing information between individual tokens. The self-attention mechanism in MHA computes the similarity score of each token with respect to all other tokens in the sequence. In particular, self-attention mechanism transform the input sequence $\mathbf{X}$ into $\text{Attn}(\mathbf{X})$ as follows:

$$\text{Attn}(\mathbf{X}) = \left(\text{Softmax} \left(\frac{(\mathbf{X}\mathbf{W}^Q)(\mathbf{X}\mathbf{W}^K)^\top}{\sqrt{d_k}} + \mathbf{M} \right) \right) \mathbf{X}\mathbf{W}^V \quad (2)$$

Here, each token generates query(Q), key(K), and value(V) vectors through the linear transformations $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V \in \mathbb{R}^{d \times d_h}$, respectively. Then, similarity scores are computed by taking the dot product of Q and K , scaled by the inverse square root of the K dimension, and passed through a softmax function to obtain the attention weights. These attention scores are used for computing a weighted sum of the V vectors, producing the output for each token. For auto-regressive models, mask $\mathbf{M} \in \mathbb{R}^{T \times T}$, with values in $\{0, -\infty\}$ with $\mathbf{M}_{i,j} = 0$ iff $i \geq j$, is used for masking the information from future tokens.

The MHA sub-block employs a self-attention mechanism across all the heads, each with its own Q , K , and V . This allows the attention heads to focus on different parts of the input sequence, capturing various aspects of the input data simultaneously. The outputs from all heads are concatenated and linearly transformed ($\mathbf{W}^O \in \mathbb{R}^{d \times d}$) to produce the final MHA output as follows:

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{Attn}_1(\mathbf{X}), \dots, \text{Attn}_H(\mathbf{X}))\mathbf{W}^O \quad (3)$$

Following the MHA sub-block, the FFN sub-block transforms each token independently. The FFN sub-blocks have a single hidden layer whose dimension is a multiple of d (e.g., $4d$ in GPT-2). Specifically, the FFN sub-block first applies a linear transformation to the input $\mathbf{X}$ using $\mathbf{W}_{\text{in}}^{\text{ffn}} \in \mathbb{R}^{d \times 4d}$, followed by a non-linear transformation using an activation function such as GELU. This is then followed by another linear transformation using $\mathbf{W}_{\text{out}}^{\text{ffn}} \in \mathbb{R}^{4d \times d}$, as follows:

$$\text{FFN}(\mathbf{X}) = (\text{GELU}(\mathbf{X}\mathbf{W}_{\text{in}}^{\text{ffn}}))\mathbf{W}_{\text{out}}^{\text{ffn}} \quad (4)$$

3.2 Threat Model and Cryptographic Protocols

Threat model for private LLM Inference We use standard two-party (2PC) client-server setting, which provides security against semi-honest (honest-but-curious) adversaries bounded by probabilistic polynomial time (Xu et al., 2025; Zhang et al., 2025a; Lu et al., 2025; Pang et al., 2024; Hou et al., 2023). Both parties follow protocol specifications but may attempt to gain additional information from their outputs about the other party’s input. In this 2PC setting (see Figure 2), the server holds the propriety LLM, and the client queries the model with a prompt. The protocols ensure that server does not know anything about the client’s input and the output of their queries, and client does not know anything about the server’s model except its architecture.

Next, we describe the cryptographic protocols used for linear and nonlinear operations during private inference. In particular, we use BumbleBee (Lu et al., 2025) private inference framework for end-to-end private inference evaluation.

Linear Operations (MatMul) For privacy-preserving matrix multiplication operations, BumbleBee leverages homomorphic encryption with a novel ciphertext compression

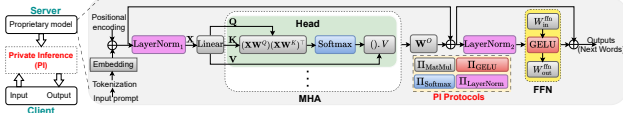


Figure 2. An illustration of threat model for private LLM inference.

strategy. The protocol implements Oblivious Linear Transform with efficient packing techniques that reduce communication costs by 80-90% compared to previous approaches. Testing on BERT-base model showed 92% less communication compared to IRON (Hao et al., 2022) and 90% less than BOLT (Pang et al., 2024). The protocol seamlessly handles both scenarios where one matrix is in plaintext and another is secret-shared, and when both matrices are secret-shared.

GELU To efficiently compute GELU activation, the protocol employs strategic polynomial approximations (with degree 3 and 6) across input ranges. The implementation optimizes branch selection through batched comparisons and leverages mixed bitwidth arithmetic, and maintains model accuracy within 1% of plaintext evaluation.

ReLU The private ReLU is implemented through a composition of secure comparison and multiplexer operations. Using optimized OT(oblivious transfer)-based comparison protocols and Boolean-to-arithmetic conversions, achieving significant efficiency gains. The protocol leverages Ferret OT (Yang et al., 2020) instead of traditional IKNP OT (Ishai et al., 2003), contributing to the overall improvement in communication efficiency while maintaining the simplicity.

LayerNorm The LayerNorm computation breaks down into optimized sub-components: secure mean calculation, variance computation using an efficient square protocol, and normalization using reciprocal square root. The implementation takes advantage of squaring operations costing half of general multiplication in secure computation. This optimization, combined with efficient reciprocal computation, shows significant improvement over IRON’s approach.

Softmax For Softmax computation, the protocol implements a numerically stable approach using max normalization. The exponential function computation is optimized specifically for negative inputs using Taylor approximation. The division operation is restructured to use reciprocal followed by multiplication, leads to 80% saving in communication costs while maintaining numerical precision (Lu et al., 2025).

3.3 Methodological Consideration

Why training from scratch to study nonlinearities? Isolating the functional role of architectural components, such as activation functions (GELU, ReLU) and LayerNorms, requires examining their impact from initialization, uninfluenced by prior optimization biases. Training from scratch provides the necessary experimental control:

1. *Capturing fundamental learning dynamics:* Nonlinear-

ities shape gradient flow and optimization trajectories from the first weight update. Training from scratch shows how these components influence convergence behavior and stability during critical early-stage learning, when attention patterns and representations first emerge.

2. *Clean ablations at feasible scales:* Starting from random initialization eliminates confounding factors from pre-trained weights, ensuring observed effects stem solely from architectural modifications. Moreover, controlled model scales (e.g., 125M parameters) enable exhaustive ablation across multiple architectural variants, which is infeasible with larger models.
3. *Limitations of fine-tuning approaches:* Fine-tuning from pre-trained checkpoints cannot reveal how nonlinearities affect early optimization since the trajectory is already determined, and learned representations mask architectural effects. Granular entropy analysis requires observing complete training trajectories from initialization.

Why perplexity is a reliable metric for evaluating LLMs at short context lengths? Perplexity (Jelinek et al., 1977) remains the standard metric for evaluating the pre-training quality, capturing token-level predictive performance, in autoregressive LLMs. However, to ensure its comparability across architectural variants, factors such as tokenizer design, vocabulary size and quality must remain consistent (Hutchins et al., 2022). Else, it can obscure the true impacts of architectural simplification. Thus, in all our experiments, we control these factors by adopting a uniform tokenizer and maintaining a fixed vocabulary *within* each dataset.

Nonetheless, perplexity does have *known limitations at very long contexts* (e.g., 32k): it weakly correlates with the true contextual understanding (Hu et al., 2024b; Fang et al., 2025). At very large context, only a small fraction of tokens (often <10 %) benefit from the extended attention window, leading to a dilution effect where perplexity mostly measures local predictability rather than long-range reasoning.

Since private LLM inference primarily focuses on smaller context lengths due to computational constraints (Zhang et al., 2025a; Lu et al., 2025; Zimerman et al., 2024; Pang et al., 2024; Gupta et al., 2024; Hou et al., 2023), we strategically use *shorter context length* (128-512), where most tokens lie well within the effective attention span. Thus, perplexity remains informative and provides a reliable basis for evaluating the effect of architectural simplifications.

Why use Shannon’s entropy to evaluate the impact of nonlinearities in LLMs? While perplexity (PPL) is the standard metric for evaluating the language modeling quality in decoder-only LLMs, it primarily reflects the model’s end-to-end behavior. As such, PPL conflates multiple error sources such as tokenization artifacts, sampling noise, and compounding effects across layers, making it *ill-suited* for diagnostic. It obscures *where* and *why* a model fails.

In contrast, Shannon’s entropy of the attention score distribution provides a fine-grained view of information flow, functional specialization, and failure modes during training. Hence, it is widely used for understanding the effect of architectural design on representation dynamics (Zhang et al., 2025b; Lee et al., 2025; Zhang et al., 2024; Nahshan et al., 2024; Vig & Belinkov, 2019; Ghader & Monz, 2017).

Below, we outline key reasons for using entropy as an interpretable metric for analyzing nonlinearities in our work:

1. *Quantifying information flow and head-specific diagnostics:* Analyzing per-head entropy statistics characterizes information flow across layers. A higher entropy corresponds to a more diffuse attention distribution, indicating exploration, whereas lower entropy signify concentration on a few key tokens, suggesting exploitation.
2. *Mechanistic specificity:* Entropy isolates the behavior of the self-attention mechanism without interference from the FFN sub-blocks, which allows us to attribute representational instability directly to the attention collapse.
3. *Actionable causality rather than correlation:* Recent studies demonstrate that attention entropy is an actionable cause—not merely a symptom—of degeneracy in transformer models. Veličković et al. (2025) proved a monotone temperature-entropy law, showing that entropy directly governs model efficiency, and Zhai et al. (2023) linked entropy collapse to the spectral norm of the score matrix, which controls gradient stability.

4 CHARACTERIZING LLM NONLINEARITY THROUGH ENTROPY DYNAMICS

Setup. To analyze the role of LLM nonlinearities, we progressively remove them from the architecture and observe their entropic dynamics at *pre-training*. Shannon’s entropy measures the uncertainty in a probability distribution, defined as $\mathbf{E}(P) = -\sum_i P(x_i) \log P(x_i)$, reflecting the amount of information needed to describe a stochastic system (Shannon, 1948; Jaynes, 1957). In Softmax-based attention mechanisms, entropy quantifies the sharpness or spread of the attention scores for each query position: higher entropy indicates a uniform distribution, while lower entropy indicates a focused attention on specific input tokens.

Let $\mathbf{A}^{(h,l)} \in \mathbb{R}^{T \times T}$ be the attention matrix of h th head in l th layer, and each element in the attention matrix ($a_{ij}^{(l,h)}$) are attention score for the i th query and j th key:

$$\mathbf{A}^{(l,h)} = [a_{ij}^{(l,h)}]_{T \times T}, \quad a_{ij}^{(l,h)} \geq 0 \text{ and } \sum_{j=1}^T a_{ij}^{(l,h)} = 1$$

This square matrix is the outcome of Softmax operation over the key length for each query position as follows

$$\mathbf{A}^{(h,l)}(\mathbf{X}) = \text{Softmax}\left(\frac{1}{\sqrt{d_k}}(\mathbf{X}\mathbf{W}^Q)(\mathbf{X}\mathbf{W}^K)^\top\right)$$

Table 1. Architectural configurations of nonlinearities in LLMs, illustrating the combinations of Softmax (SM), LayerNorm (LN), GELU (G), and ReLU (R) functions (see Eq. 1, 2, 3 and 4).

Abbreviation	Architectural configuration
SM + LN + G	$\mathbf{X}_{\text{out}} = \text{FFN}_{\text{GELU}}(\text{LayerNorm}_2(\text{MHA}(\text{Attn}_{\text{Softmax}}(\text{LayerNorm}_1(\mathbf{X}_{\text{in}}))))$
SM + LN + R	$\mathbf{X}_{\text{out}} = \text{FFN}_{\text{ReLU}}(\text{LayerNorm}_2(\text{MHA}(\text{Attn}_{\text{Softmax}}(\text{LayerNorm}_1(\mathbf{X}_{\text{in}}))))$
SM + LN	$\mathbf{X}_{\text{out}} = \text{FFN}_{\text{Identity}}(\text{LayerNorm}_2(\text{MHA}(\text{Attn}_{\text{Softmax}}(\text{LayerNorm}_1(\mathbf{X}_{\text{in}}))))$
SM + G	$\mathbf{X}_{\text{out}} = \text{FFN}_{\text{GELU}}(\text{MHA}(\text{Attn}_{\text{Softmax}}(\mathbf{X}_{\text{in}})))$
SM + R	$\mathbf{X}_{\text{out}} = \text{FFN}_{\text{ReLU}}(\text{MHA}(\text{Attn}_{\text{Softmax}}(\mathbf{X}_{\text{in}})))$
SM	$\mathbf{X}_{\text{out}} = \text{FFN}_{\text{Identity}}(\text{MHA}(\text{Attn}_{\text{Softmax}}(\mathbf{X}_{\text{in}})))$

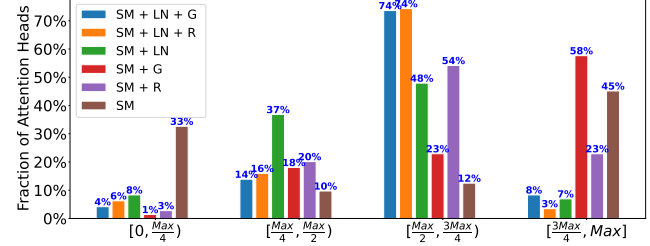


Figure 3. Distribution of attention heads (%) across entropy ranges for different model configurations (Table 1) trained from scratch, showing the concentration of heads in specific entropy intervals.

Following Zhai et al. (2023), we compute the average entropy across all query positions within an attention head to obtain its headwise entropy. Entropy $\mathbf{E}^{(l,h)}$ for the h th head in l th layer of an attention matrix is given by:

$$\mathbf{E}^{(l,h)} = -\frac{1}{T} \sum_{i=1}^T \sum_{j=1}^T a_{ij}^{(l,h)} \log(a_{ij}^{(l,h)}),$$

$$\text{where } a_{ij}^{(l,h)} = \frac{\exp\left(\frac{1}{\sqrt{d_k}}(\mathbf{X}_i \mathbf{W}^Q)(\mathbf{X}_j \mathbf{W}^K)^\top\right)}{\sum_{k=1}^T \exp\left(\frac{1}{\sqrt{d_k}}(\mathbf{X}_i \mathbf{W}^Q)(\mathbf{X}_k \mathbf{W}^K)^\top\right)}$$

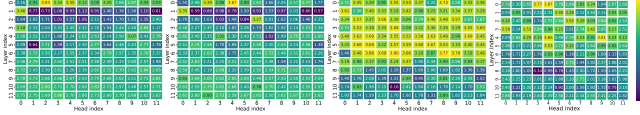
Well-behaved entropy distribution for LLMs. First, we analyze the headwise entropy distribution of baseline architecture with GELU and ReLU in the FFN, i.e., configurations SM + LN + G and SM + LN + R, respectively. We find that the majority of heads ($\approx 90\%$) possess entropy values between $\frac{\text{max}}{4}$ and $\frac{3\text{max}}{4}$, where max is maximum observed entropy value among all heads (see Figure 3). This concentration in the mid-entropy range, while avoiding extremes, shows a well-behaved distribution, providing a benchmark for assessing the impact of nonlinearities on model behavior.

Entropic overload in LLMs. We observed that in the absence of crucial nonlinearities, a disproportionately large fraction of the attention heads exhibit higher entropy values (between $\frac{3\text{max}}{4}$ and max). We term this phenomenon as entropic overload and hypothesize that this imbalance results in *under-utilization* of multi-head attention (MHA), as too many heads engaged in exploration, impairing the model’s ability to learn diverse and specialized attention patterns.

To better understand this behavior, we analyze how entropy evolves throughout training. In models with all nonlinearities intact, attention heads typically start with high entropy, reflecting an initial phase of exploration, and gradually tran-

Table 2. Performance comparison of baseline and normalization-free GPT-2 (12L, 12H, 768d) and Pythia-70M (6L, 8H, 512d) models, each with GELU and ReLU variants, trained on CodeParrot. While GELU outperforms ReLU in baseline models, the normalization-free models exhibit the *opposite trend*.

	GPT-2 ($T=128$)		Pythia-70M ($T=128$)		Pythia-70M ($T=256$)	
	Eval PPL	+ Δ (%)	Eval PPL	+ Δ (%)	Eval PPL	+ Δ (%)
SM+LN+G	2.69	0.00	3.51	0.00	3.05	0.00
SM+LN+R	2.76	2.53	3.59	2.22	3.11	1.73
SM+G	3.20	18.92	4.09	16.35	3.57	16.87
SM+R	2.94	9.20	3.74	6.36	3.27	7.17



(a) SM + LN + G (b) SM + LN + R (c) SM + G (d) SM + R

Figure 4. Entropy heatmaps of GPT-2 with GELU and ReLU in the FFN (a, b) and their normalization-free variants (c, d). Without LayerNorm, *GELU causes significantly higher entropic overload*, situation toward a balance between exploration and exploitation (see Figure 12a and 12b). However, when key nonlinearities are removed, this balance breaks down and any attention heads in the early layers remain stuck in persistently higher entropy states (see Figure 12d), which impedes learning and ultimately degrades model performance.

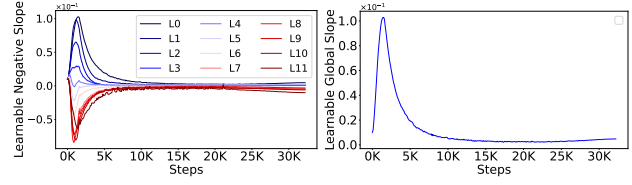
Entropy collapse in LLMs. This is characterized by extremely-smaller (near-zero) entropy values and recognized as a key indicator of training instability in transformer architectures (Zhai et al., 2023; He et al., 2024).

4.1 Activation Functions in Normalization-Free LLMs

While GELU is typically favored in transformer models for its smoothness and optimization benefits, we observe the *opposite trend* in normalization-free settings: ReLU outperforms GELU by achieving a **8.2%** lower perplexity, which is consistent with the normalization-free Pythia models across various context lengths (Table 2).

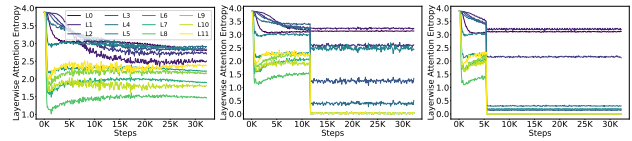
Observation 1: In normalization-free LLM, ReLU outperforms GELU due to lesser entropic overload. Figure 3 shows that 58% of attention heads in the normalization-free GELU (SM+G) model have entropy values in the range $\frac{3\max}{4}$ and max, compared to only 23% in the normalization-free ReLU model (SM+R). Moreover, very few heads in the latter approach maximum entropy, unlike GELU variant (see yellow regions in Figure 4d & 4c). This over-concentration of high-entropy heads in SM+G suggests a lack of attention diversity and specialization, which we believe is a key reason for its higher perplexity compared to SM+R.

Observation 2: Normalization-free models with learnable activations naturally converge to ReLU-behavior To investigate the *inductive bias* favoring ReLU over GELU in normalization-free settings, we parameterize the Leaky



(a) Layerwise learnable slope (b) Global learnable slope

Figure 5. Normalization-free GPT-2 with LeakyReLU having learnable negative slope. In both settings, (a) layerwise and (b) global, slope values converge toward zero during training, indicating a preference for zero negative slope—A ReLU-like behavior.



(a) Learnable slope (b) Fixed slope $5e-2$ (c) Fixed slope $1e-1$

Figure 6. Entropy dynamics in normalization-free GPT-2 (125M) with LeakyReLU having: learnable negative slopes (a) and fixed slopes (b,c). While fixed slopes lead to earlier entropy collapse, learnable configuration mimic the stable entropy profile of ReLU.

ReLU activation with a learnable negative slope and evaluate two configurations: (1) layer-wise, where each layer has an independently learnable slope parameter, and (2) global, where a single slope is shared across all layers. Results in Figure 5 shows that, in the layer-wise setting, early layers initially learn positive slopes while deeper layers exhibit negative ones; however, all slope parameters converge toward zero. In the global setting, the shared slope exhibits an initial drift toward positive values before approaching zero.

Further, we analyzed the entropy dynamics under learnable slope configurations and compared them to fixed negative slope settings in Leaky ReLU (with slopes of $5e-2$ and $1e-1$). Interestingly, the learnable configurations exhibit entropy dynamics that closely resembles those of normalization-free ReLU (Figure 6). In contrast, fixed-slope Leaky ReLU exhibits a clear trend: larger negative slopes induce earlier entropy collapse. This suggests that increasing the negative slope reduces the window of stable training. These findings further reinforce the inductive bias toward ReLU-like activations in normalization-free LLMs

4.2 Selective Removal of Activations in FFNs

While completely removing FFN activations degrades performance by **+25.6%** (3.376 vs 2.688 PPL, Table 3), this degradation may not be uniform across layers, motivating us to explore selective removal strategies. Specifically, we ask: *do FFN layers exhibit location-dependent criticality?*

To this end, we explored three strategies: (1) 25% activation, where we divide the network into four stages and remove activations from three; (2) 50% activation, where activations

Table 3. Partial removal of FFN activations: perplexity (PPL) and selective FFN layers with activations in GPT-2 (125M) variants. Removing activations from deeper layers results in worse PPL.

Activation	Config 1	Config 2	Config 3	Config 4
Full-GELU		PPL: 2.688		
Full-ReLU		PPL: 2.757		
50%	PPL: 2.903	2.889	2.885	2.877
GELU	L(0,1,2,3,4,5)	L(6,7,8,9,10,11)	L(0,2,4,6,8,10)	L(1,3,5,7,9,11)
50%	PPL: 2.961	2.943	2.944	2.931
ReLU	L(0,1,2,3,4,5)	L(6,7,8,9,10,11)	L(0,2,4,6,8,10)	L(1,3,5,7,9,11)
25%	PPL: 3.126	3.071	3.061	3.054
GELU	L(0,1,2)	L(3,4,5)	L(6,7,8)	L(9,10,11)
25%	PPL: 3.147	3.097	3.095	3.093
ReLU	L(0,1,2)	L(3,4,5)	L(6,7,8)	L(9,10,11)
Fractional	PPL: 2.875	3.040	3.161	3.243
GELU	50% neurons	25% neurons	12.5% neurons	6.25% neurons
No-Acts		PPL: 3.376		

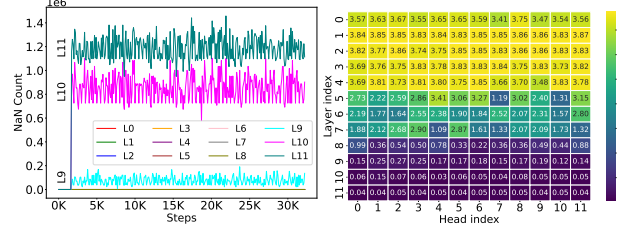
are removed from alternating layers or half the network; and (3) fractional GELU, where GELU is applied to only a fraction of neurons within each layer.

Observations 3: Deeper FFNs require activations more critically than early layers. Across all partial configurations, removing activations from deeper layers consistently leads to higher perplexity, indicating that deeper FFNs require activations more than the early layers (Table 3). For instance, 25% GELU in layers $L0-L2$ yields 3.126 PPL versus 3.054 PPL when placed in $L9-L11$. This observation is consistent for both GELU and ReLU activations. Future private LLM architectures could exploit this asymmetry for reducing the overheads of FFN activations.

4.3 Softmax-only Architecture

Having shown the viability of selective activation removal, we now examine the extreme minimal-nonlinearity configuration: eliminating all activations and normalizations, except the Softmax in attention sub-block. This allows us to verify architectural and algorithmic solutions to linearize private LLMs under the most constrained nonlinear regime.

Observation 4: The softmax-only model exhibits severe entropic overload in the early layers and entropy collapse in the deeper layers. When FFN nonlinearity is removed in a normalization-free architecture, leaving Softmax as the sole source of nonlinearity and the FFN purely linear, the resulting design is referred to as a Softmax-only model. However, training such a model is challenging, as loss values in deeper layers quickly diverge to NaNs (Figure 7a). The entropic analysis further reveals that the deeper layers suffers from entropy collapse, marked by extremely low entropy values (see blue regions in Figure 7b), a known indicator of training instability (Zhai et al., 2023). Moreover, 45% of total heads exhibit entropy values in the range of $\frac{3\max}{4}$ to max, with most close to the maximum value (Figure 3) in the early layers, indicating severe entropic overload.



(a) Layerwise NaNs (b) Entropy heatmap
Figure 7. Training collapses in softmax-only GPT-2 model.

5 AERO

We introduce AERO, an entropy-guided architectural framework that remove nonlinearities and reduces FLOPs through principled refinements in LLM architectures, without destabilizing them. Further, we propose a novel entropy regularization technique to prevent entropic overload, by aligning the model’s entropy dynamics closer to the well-behaved attention entropy distribution in MHA.

5.1 Inference-efficient Substitute for LayerNorm

To address training instability, prior work has predominantly relied on LayerNorm applied to various parts of the network, such as QK-LayerNorm (Dehghani et al., 2023; Wortsman et al., 2024; Muennighoff et al., 2024) and FFN-LayerNorm (Rybakov et al., 2024). Since LayerNorm requires expensive inverse-square-root operations during inference (Hou et al., 2023), we explore the weight and other static normalization alternatives to avoid nonlinear computations at inference.

Normalizing weights in FFN linear layers prevent entropy collapse and stabilizes training. We find that weight normalization (Salimans & Kingma, 2016) and spectral normalization (Miyato et al., 2018) offer inference-efficient alternatives to LayerNorm by normalizing weights rather than activations. When applied *strategically*, they can effectively prevent entropy collapse in the deeper layers of Softmax-only LLMs, without incurring additional inference overhead in privacy-preserving computation settings. Notably, the effectiveness of these methods varies with the targeted linear layers: applying normalization within the attention sub-block results in *higher* perplexity compared to its application within the FFN (see Appendix C).

Further, we employ a more simpler approach to stabilize the training of Softmax-only models. During training FFN sub-blocks learn their individual (down)scaling factors for FFN output and its residual output, as follows (see Eq. 1):

$$\mathbf{X}_{\text{out}} = \beta \hat{\mathbf{X}}_{\text{SA}} + \frac{1}{\alpha} (\text{FFN}^{\text{SM}}(\mathbf{X}_{\text{SA}})) \text{ where } \alpha, \beta \in \mathbb{R}^L \quad (5)$$

Figure 8 illustrates how these normalization techniques stabilize training by preventing entropy collapse in deeper layers of softmax-only models. Learnable scaling in FFNs exhibits better performance (lowest perplexity) compared to weight and spectral normalization (Table 4).

Table 4. Perplexity comparison of inference-efficient static normalization methods employed in FFNs of Softmax-only GPT-2.

	WeightNorm	SpectralNorm	ScaledFFN
Eval PPL	3.640	3.624	3.478

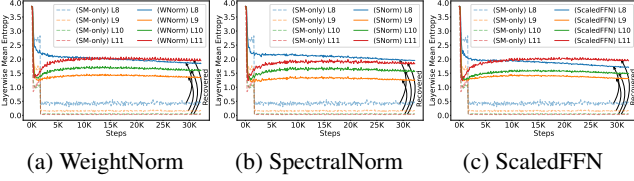


Figure 8. Inference-efficient static normalization methods mitigate entropy collapse in the deeper layers of a softmax-only models, showing its utility for LayerNorm substitute for private LLMs.

5.2 Entropy Regularization

Challenges in designing entropy regularization schemes to prevent entropic overload. Previous entropy regularization approaches have primarily aimed at penalizing low-entropy predictions (Setlur et al., 2022; Pereyra et al., 2017), based on the principle of maximum entropy (Jaynes, 1982). Recently, He et al. (2024) introduced entropy regularization in LLM to address extremely low entropy values.

However, our goal is to regularize higher entropy values, which introduces two key challenges: (1) *Head specialization*: Since each attention head captures different aspects of the input, the regularization strength needs to be adjusted for each head individually. (2) *Over-regularization*: Some heads naturally exhibit higher entropy, thus, blanket penalization of all higher entropy values could be detrimental, requiring a more flexible approach.

Key design principles for entropy regularization. To address aforementioned challenges, our regularization scheme (see Algorithm 1) is guided by following principles:

- *Balanced entropy distribution with parameterized attention matrix*: Inspired by Miller et al. (1996), which used temperature parameter as a Lagrangian multiplier to control the entropy of a stochastic system, we parameterized the attention matrix by a learnable temperature $t \in \mathbb{R}^{H \times T}$ for each softmax operation, allowing the model to adjust the sharpness of the attention scores. A higher temperature ($t > 1$) diffuses attention scores and increases the entropy, while a lower temperature ($t < 1$) sharpen attention scores and reduces the entropy.
- *Dynamic thresholds with head-specific adaptation*: To adapt the regularization strength based on the characteristics of each attention head (Voita et al., 2019), we use headwise learnable threshold parameter $\text{reg_threshold_weights} \in \mathbb{R}^H$. Consequently, the threshold for each head is computed as a learnable fraction of the maximum value of entropy ($\text{reg_threshold_weights} \times E_{\max}$), providing the fine-grained control (see Algorithm 1, line #1).

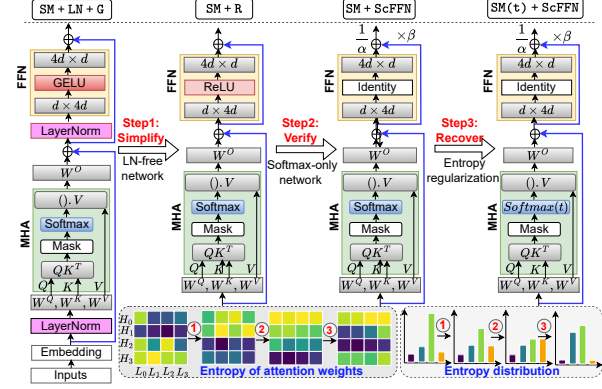


Figure 9. Overview of the AERO framework for efficient private LLM design. Step 1 simplifies the architecture by removing LayerNorm and substituting GELU with ReLU in FFNs. Step 2 validates our approach through softmax-only design. Step 3 employs entropy regularization to penalize extreme entropy values.

- *Tolerance margin to prevent over-regularization*: To prevent over-regularization, we allow small deviations from the respective thresholds. Thus, a penalty is imposed only if the deviation from the threshold exceeds the tolerance margin, which is set as a fraction of $E_{\max}$ using the hyper-parameter γ (see Algorithm 1, line #1).

$$\text{penalty}^{(l,h)} = \begin{cases} \left(\text{deviation}^{(l,h)} \right)^2 & \text{iff } |\text{deviation}^{(l,h)}| > \gamma E_{\max} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

The deviation from threshold is computed as $\text{deviation}^{(l,h)} = E^{(l,h)}(t) - \theta^{(l,h)} E_{\max}$, where $\theta^{(l,h)}$ is $\text{reg_threshold_weights}$. The hyper-parameter γ ensures that the model is not excessively penalized for minor deviations from the desired entropy threshold, which could impede its capacity to learn effectively. This careful calibration between stringent regularization and desired flexibility improves the model’s robustness while maintaining its adaptability to various input distributions.

- *Maximum entropy reference*: We set $E_{\max} = \log(T)$ as a reference for thresholds and tolerance margins, ensuring consistency across layers and heads.

5.3 Putting it All Together

The AERO framework addresses private inference inefficiencies through targeted architectural refinements and algorithmic innovations. Given an input LLM, **Step 1** tackles communication overhead by identifying by identifying SM+R, a configuration that strategically removes LayerNorm and substitute GELU with ReLU in FFNs. **Step 2** validates these choices through extreme stress testing: the softmax-only architecture which highlights the practical utility of architectural simplification. Finally, **Step 3** provides the critical algorithmic component, adaptive entropy regularization that maintains attention diversity by penalizing extreme distributions in the absence of nonlinearities.

6 EVALUATION

System setup We use a SecretFlow setup (Lu et al., 2025) with the client and server simulated on two physically separate machines, each equipped with an AMD EPYC 7502 server with specifications of 2.5 GHz, 32 cores, and 256 GB RAM. We measure the *end-to-end* PI latency, including input embeddings and final output (vocabulary projection) layers, in WAN setting (bandwidth:100Mbps, latency:80ms), simulated using Linux Traffic Control (tc) commands. The number of threads is set to 32. Following (He & Hofmann, 2024; Stanić et al., 2023; Geiping & Goldstein, 2023), all the models are trained on a single RTX 3090 GPU.

Models and datasets We conducted experiments on GPT-2 models (12 and 18 layers) and Pythia, training them from scratch on the CodeParrot and Languini book datasets—standard benchmarks for LLM evaluation (He & Hofmann, 2024; He et al., 2024). CodeParrot dataset is sourced from 20 million Python files, contains 8 GB of files, a total of 2.1 billion training tokens. We use a tokenizer with a vocabulary of 50K and train with context lengths of 128 and 256. The Languini book dataset includes 84.5 GB of text, and 23.9 billion tokens with a WikiText-trained vocabulary of 16,384, and we train with context of 512 tokens.

Training hyperparameters For pre-training on the CodeParrot dataset, we adopt the training settings from (He & Hofmann, 2024). Similarly, for training on the Languini dataset, we follow the settings from (Stanić et al., 2023). These settings remain consistent across all architectural variations to accurately reflect the impact of the architectural changes. When applying entropy regularization on the CodeParrot dataset, we initialize the learnable temperature to $1e-2$ and set λ to $1e-5$. For the Languini dataset, the temperature is initialized to $1e-1$, and λ is set to $5e-5$.

6.1 Effectiveness of Entropy Regularization

Entropy regularization prevents entropic overload While weight/spectral normalization, and learnable scaling normalization in FFNs effectively prevent entropy collapse in the deeper layers and stabilize the training of Softmax-only models, they *fail to address the issue of entropic overload*, (Figure 10). In contrast, the entropy regularization scheme penalizes the model to avoid extreme entropy values during training, resulting in a more balanced distribution. Hence, it complements the training stabilizing methods by further mitigating entropic overload in the early layers (Figure 10), improving the attention heads utilization and perplexity.

Learnable thresholds help preserve the attention head diversity and tolerance margins prevent over-regularization Figure 11a depicts threshold parameters (reg_threshold_weights) of entropy regularization scheme employed in Softmax-only models, after the model

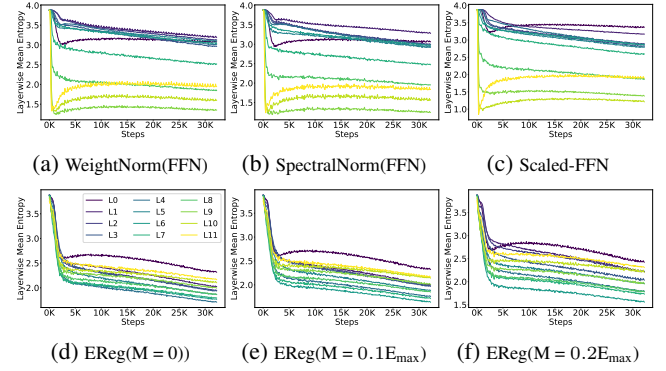
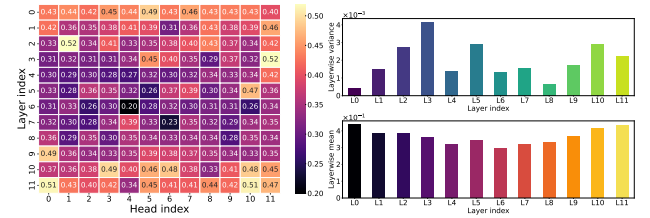


Figure 10. Entropy dynamics of a Softmax-only GPT-2 model trained from scratch on 2.1B tokens. **Top:** Weight normalization, spectral normalization, and learnable scaling in FFNs *alone* do not prevent entropic overload, indicated by persistent higher entropy in early layers. **Bottom:** Entropy regularization mitigates this overload, while a non-zero tolerance margin avoids over-regularization.



(a) Learned threshold weights (b) Layerwise mean and variance

Figure 11. Learned threshold weights in entropy regularization (Eq. 6) in a trained softmax-only GPT-2 model: Attention heads adaptively learned nonuniform threshold across heads and layers.

has been trained. They exhibit significant variability, both across layers and within individual heads of each layers, which reflects the model’s ability to dynamically adjust the regularization strength in tailored to the head-specific roles.

Figures 10 (bottom) shows the effect of nonzero tolerance margins in entropy regularization. Higher margins allow a small yet crucial fraction of early-layer attention heads to attain higher entropy values, desirable for well-behaved entropy distribution. Specifically, as shown in Figure 15, γ increases from 0 to 0.15, only 0.7% of attention heads fall in the highest entropy range, but this fraction rises to 2.08%, 3.47%, and 6.25% at $\gamma = 0.20, 0.25$, and 0.30 , respectively.

6.2 Practical Utility of AERO for Private LLM Design

Efficiency gains without performance loss The practical application of AERO hinges on the efficiency gain through nonlinearity reduction while recovering performance loss through entropy regularization. Table 5 shows that AERO enables the SM+R configuration to achieve $3.41\times$ communication reduction and $1.42\times$ speedup while fully recovering performance through entropy regularization (PPL: 2.66 vs 2.69 baseline). However, removing entropy regularization significantly drops the performance (2.94 PPL). Table 6

Table 5. Results, and comparison against SOTA (He & Hofmann, 2024), when GPT-2 ($L=12$, $H=12$, $d=768$) model is trained from scratch on CodeParrot dataset with context length 128.

	Network Arch.	PPL	#Nonlinear Ops	Comm. (GB)	Lat. (min.)	Savings	
						Comm.	Lat.
Baseline	SM + LN + G	2.69	SM: $144 \times \mathbb{R}^{128 \times 128}$ LN: $24 \times \mathbb{R}^{128 \times 768}$ G: $12 \times \mathbb{R}^{128 \times 3072}$	25.32	8.21	1×	1×
	SM + LN + R	2.76	SM: $144 \times \mathbb{R}^{128 \times 128}$ LN: $24 \times \mathbb{R}^{128 \times 768}$ R: $12 \times \mathbb{R}^{128 \times 3072}$	9.44	6.06	2.68×	1.35×
Norm-free	SM + G	3.20	SM: $144 \times \mathbb{R}^{128 \times 128}$ G: $12 \times \mathbb{R}^{128 \times 3072}$	23.31	7.86	1.09×	1.04×
	SM + R	2.94	SM: $144 \times \mathbb{R}^{128 \times 128}$ R: $12 \times \mathbb{R}^{128 \times 3072}$	7.43	5.78	3.41×	1.42×
SOTA	SM + G	2.93	SM: $144 \times \mathbb{R}^{128 \times 128}$ G: $12 \times \mathbb{R}^{128 \times 3072}$ LN: $1 \times \mathbb{R}^{128 \times 768}$	23.19	7.40	1.09×	1.11×
	SM + R	3.03	SM: $144 \times \mathbb{R}^{128 \times 128}$ R: $12 \times \mathbb{R}^{128 \times 3072}$ LN: $1 \times \mathbb{R}^{128 \times 768}$	7.30	5.55	3.47×	1.48×
AERO	EReg(SM + G)	3.07	SM: $144 \times \mathbb{R}^{128 \times 128}$ G: $12 \times \mathbb{R}^{128 \times 3072}$	23.31	7.86	1.09×	1.04×
	EReg(SM + R)	2.66	SM: $144 \times \mathbb{R}^{128 \times 128}$ R: $12 \times \mathbb{R}^{128 \times 3072}$	7.43	5.78	3.41×	1.42×

Table 6. ReLU exhibits higher sensitivity to tolerance margins than GELU in LayerNorm-free GPT-2 models (8% vs 1.5% variation)

	EReg(SM+G)			EReg(SM+R)		
	TM=0	TM=0.10	TM=0.20	TM=0	TM=0.10	TM=0.20
Eval PPL	3.113	3.100	3.067	2.924	2.658	2.886

shows the mechanism driving this performance recovery. ReLU variants exhibit strong sensitivity to tolerance margins, improving from 2.924 to 2.688 PPL (at TM = 0.10), an 8% gain, whereas GELU reduces PPL only by 1.5%.

When compared to state-of-the-art LayerNorm-free method (He & Hofmann, 2024), AERO achieves comparable communication savings without retaining any LayerNorm layers and delivers **14%** lower perplexity (2.66 vs 3.03). Notably, their ReLU variant yields higher perplexity, suggesting that the underlying optimization is not tailored for PI efficiency.

6.3 Scalability of AERO

Scalability across training scales We perform experiments on the Languini dataset across varying token counts (1.2B to 4.8B). We observed consistent improvements in perplexity across the training scale (Table 7), when entropy regularization is employed on most-stringent nonlinearity configuration (Softmax-only), whereas SOTA exhibits training instability even at the lower training scale (1.2B).

Scalability across model depths and context lengths To demonstrate the scalability of AERO, particular the our loss recovery approach, entropy regularization, we employ AERO on Softmax-only GPT-2 variants across multiple depths and context lengths (Table 8). While the Softmax-only models show severe performance degradation compared to their baselines, entropy regularization consistently improves perplexity by **6%–8%**. However, SOTA often

Table 7. GPT-2 ($L=12$, $H=12$, $d=768$) trained from scratch on Languini (Stanić et al., 2023) dataset with context length 512. NaNs indicate training instability in SOTA (He & Hofmann, 2024).

	Network Arch.	Eval PPL			#Nonlinear Ops	Comm. (GB)	Lat. (min.)
		1.2B	2.4B	4.8B			
Baseline	SM + LN + G	25.71	23.32	21.29	SM: $144 \times \mathbb{R}^{512 \times 512}$ LN: $24 \times \mathbb{R}^{512 \times 768}$ G: $12 \times \mathbb{R}^{512 \times 3072}$	145.24	30.74
	SM + LN + R	26.06	23.55	21.58	SM: $144 \times \mathbb{R}^{512 \times 512}$ LN: $24 \times \mathbb{R}^{512 \times 768}$ R: $12 \times \mathbb{R}^{512 \times 3072}$	81.71	23.54
SOTA	SM + ScFFN	NaNs	NaNs	NaNs	SM: $144 \times \mathbb{R}^{512 \times 512}$ LN: $1 \times \mathbb{R}^{512 \times 768}$	72.10	21.56
AERO	SM + ScFFN	33.91	31.12	28.89	SM: $144 \times \mathbb{R}^{512 \times 512}$	71.76	21.51
	EReg(SM(t) + ScFFN)	31.65	28.74	26.61	SM: $144 \times \mathbb{R}^{512 \times 512}$	71.76	21.51

Table 8. Comparison of AERO with SOTA (He & Hofmann, 2024) across model depths and context sizes using Softmax-only variants, trained on CodeParrot dataset. NaNs indicate training instability. Refer to Table 8 for baseline performance and their PI overheads

Network Arch.		PPL	#Nonlinear Ops	Comm. (GB)	Lat. (min.)	Savings	
						Comm.	Lat.
GPT-2 ($L=12, H=12, d=768$), $T=128$; Baseline PPL: GELU=2.69, ReLU=2.76							
AERO SOTA	SM + ScFFN	4.00	SM: $144 \times \mathbb{R}^{128 \times 128}$ LN: $1 \times \mathbb{R}^{128 \times 768}$	6.83	5.31	$3.71 \times$	$1.55 \times$
	SM + ScFFN	3.50	SM: $144 \times \mathbb{R}^{128 \times 128}$	6.95	5.68	$3.64 \times$	$1.45 \times$
	EReg(SM(t) + ScFFN)	3.24	SM: $144 \times \mathbb{R}^{128 \times 128}$	6.43	4.76	$3.94 \times$	$1.72 \times$
GPT-2 ($L=12, H=12, d=768$), $T=256$; Baseline PPL: GELU=2.56, ReLU=2.63							
AERO SOTA	SM + ScFFN	3.47	SM: $144 \times \mathbb{R}^{256 \times 256}$ LN: $1 \times \mathbb{R}^{256 \times 768}$	21.52	11.42	$2.72 \times$	$1.45 \times$
	SM + ScFFN	3.04	SM: $144 \times \mathbb{R}^{256 \times 256}$	21.77	11.91	$2.69 \times$	$1.39 \times$
	EReg(SM(t) + ScFFN)	2.96	SM: $144 \times \mathbb{R}^{256 \times 256}$	20.72	10.45	$2.82 \times$	$1.59 \times$
GPT-2 ($L=18, H=12, d=768$), $T=128$; Baseline PPL: GELU=2.35, ReLU=2.41							
AERO SOTA	SM + ScFFN	NaNs	SM: $216 \times \mathbb{R}^{128 \times 128}$ LN: $1 \times \mathbb{R}^{128 \times 768}$	9.39	6.75	$3.96 \times$	$1.60 \times$
	SM + ScFFN	3.26	SM: $216 \times \mathbb{R}^{128 \times 128}$	9.62	7.23	$3.86 \times$	$1.49 \times$
	EReg(SM(t) + ScFFN)	3.15	SM: $216 \times \mathbb{R}^{128 \times 128}$	8.83	6.07	$4.21 \times$	$1.77 \times$

exhibits training instability under these conditions. This performance gap can be attributed to the critical role of MHA in model pre-training performance, particularly in the absence of FFN operations (Lu et al., 2024). We hypothesize that SOTA’s aggressive optimization of attention FLOPs, unlike AERO’s more balanced approach, leads to both inferior predictive performance and training instability.

7 CONCLUSION

We perform an information-theoretic analysis for understanding the role of nonlinearities in pre-training learning dynamics, and developed entropy-guided framework (AERO) to strategically remove the costlier nonlinearities, achieving $3.41\times$ communication reduction and $1.42\times$ speedup without incurring performance loss. Although our study focuses on sub-billion parameter LLMs, the insights are likely to be relevant for larger models as well (Wortsman et al., 2024).

Limitations. This work focuses on pre-training performance through perplexity as a primary metric, and does not evaluate transfer learning or few-shot learning capabilities. Future work will extend AERO to large-scale LLMs with broader experimental evaluation.(see Appendix H).

REFERENCES

- Ahmed, Z., Le Roux, N., Norouzi, M., and Schuurmans, D. Understanding the impact of entropy on policy optimization. In *International conference on machine learning (ICML)*, 2019.
- Bao, H., Hataya, R., and Karakida, R. Self-attention networks localize when QK-eigenspectrum concentrates. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Bondarenko, Y., Nagel, M., and Blankevoort, T. Quantizable transformers: Removing outliers by helping attention heads do nothing. In *Advances in Neural Information Processing Systems*, 2023.
- Brody, S., Alon, U., and Yahav, E. On the expressivity role of layernorm in transformers’ attention. In *Findings of the Association for Computational Linguistics (ACL)*, 2023.
- Cancedda, N. Spectral filters, dark signals, and attention sinks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Carlini, N., Paleka, D., Dvijotham, K. D., Steinke, T., Hayase, J., Cooper, A. F., Lee, K., Jagielski, M., Nasr, M., Conmy, A., et al. Stealing part of a production language model. In *International Conference on Machine Learning (ICML)*, 2024.
- Chen, D., Zhang, Y., Kundu, S., Li, C., and Beerel, P. A. Rna-vit: Reduced-dimension approximate normalized attention vision transformers for latency efficient private inference. In *IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023.
- Cheng, X., Chen, Y., and Sra, S. Transformers implement functional gradient descent to learn non-linear functions in context. In *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- Dehghani, M., Djolonga, J., Mustafa, B., Padlewski, P., Heek, J., Gilmer, J., Steiner, A. P., Caron, M., Geirhos, R., Alabdulmohsin, I., et al. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning (ICML)*, 2023.
- Dettmers, T. and Zettlemoyer, L. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning (ICML)*, 2023.
- Dhyani, N., Mo, J., Yubeaton, P., Cho, M., Joshi, A., Garg, S., Reagen, B., and Hegde, C. Privit: Vision transformers for fast private inference. In *Transactions on Machine Learning Research (TMLR)*, 2024.
- Fang, L., Wang, Y., Liu, Z., Zhang, C., Jegelka, S., Gao, J., Ding, B., and Wang, Y. What is wrong with perplexity for long-context language modeling? In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Geiping, J. and Goldstein, T. Cramming: Training a language model on a single gpu in one day. In *International Conference on Machine Learning (ICML)*, 2023.
- Ghader, H. and Monz, C. What does attention in neural machine translation pay attention to? In *Proceedings of the The 8th International Joint Conference on Natural Language Processing*, 2017.
- Gu, X., Pang, T., Du, C., Liu, Q., Zhang, F., Du, C., Wang, Y., and Lin, M. When attention sink emerges in language models: An empirical view. *arXiv preprint arXiv:2410.10781*, 2024.
- Gu, Y., Dong, L., Wei, F., and Huang, M. Minillm: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2023.
- Gupta, K., Jawalkar, N., Mukherjee, A., Chandran, N., Gupta, D., Panwar, A., and Sharma, R. SIGMA: secure GPT inference with function secret sharing. In *Proceedings on Privacy Enhancing Technologies (PETs)*, 2024.
- Gurnee, W., Nanda, N., Pauly, M., Harvey, K., Troitskii, D., and Bertsimas, D. Finding neurons in a haystack: Case studies with sparse probing. In *Transactions on Machine Learning Research (TMLR)*, 2023.
- Hao, M., Li, H., Chen, H., Xing, P., Xu, G., and Zhang, T. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems*, 2022.
- He, B. and Hofmann, T. Simplifying transformer blocks. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- He, B., Martens, J., Zhang, G., Botev, A., Brock, A., Smith, S. L., and Teh, Y. W. Deep transformers without shortcuts: Modifying self-attention for faithful signal propagation. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- He, B., Noci, L., Paliotta, D., Schlag, I., and Hofmann, T. Understanding and minimising outlier features in neural network training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Hou, X., Liu, J., Li, J., Li, Y., Lu, W.-j., Hong, C., and Ren, K. Ciphergpt: Secure two-party gpt inference. *Cryptology ePrint Archive*, 2023.

- Hsieh, C.-Y., Li, C.-L., Yeh, C.-K., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- Hu, J. Y.-C., Chang, P.-H., Luo, H., Chen, H.-Y., Li, W., Wang, W.-P., and Liu, H. Outlier-efficient hopfield layers for large transformer-based models. In *Forty-first International Conference on Machine Learning (ICML)*, 2024a.
- Hu, Y., Huang, Q., Tao, M., Zhang, C., and Feng, Y. Can perplexity reflect large language model’s ability in long text understanding? In *The Second Tiny Papers Track at ICLR 2024*, 2024b.
- Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. Block-recurrent transformers. In *Advances in neural information processing systems (NeurIPS)*, 2022.
- Ishai, Y., Kilian, J., Nissim, K., and Petrank, E. Extending oblivious transfers efficiently. In *Annual International Cryptology Conference (CRYPTO)*, 2003.
- Jagatap, G., Joshi, A., Chowdhury, A. B., Garg, S., and Hegde, C. Adversarially robust learning via entropic regularization. In *Frontiers in artificial intelligence*, 2022.
- Jaynes, E. T. Information theory and statistical mechanics. *Physical review*, 1957.
- Jaynes, E. T. On the rationale of maximum-entropy methods. In *Proceedings of the IEEE*, 1982.
- Jelinek, F., Mercer, R. L., Bahl, L. R., and Baker, J. K. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, 1977.
- Jo, J.-y. and Myaeng, S.-H. Roles and utilization of attention heads in transformer-based neural language models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- Joudaki, A., Daneshmand, H., and Bach, F. On the impact of activation and normalization in obtaining isometric embeddings at initialization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Knott, B., Venkataraman, S., Hannun, A., Sengupta, S., Ibrahim, M., and van der Maaten, L. Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems*, 2021.
- Ko, J., Kim, S., Chen, T., and Yun, S.-Y. Distillm: Towards streamlined distillation for large language models. In *International Conference on Machine Learning (ICML)*, 2024.
- Kovaleva, O., Kulshreshtha, S., Rogers, A., and Rumshisky, A. Bert busters: Outlier dimensions that disrupt transformers. In *Findings of the Association for Computational Linguistics (ACL-IJCNLP)*, 2021.
- Lauren, G. and Knight, W. Chatgpt can now talk to you—and look into your life. <https://www.wired.com/story/chatgpt-can-now-talk-to-you-and-look-into-your-life/>, 2023.
- Lee, N., Kim, Y., Oh, M., Kim, S., Koo, J. W., Jo, H., and Lee, J. Mitigating attention localization in small scale: Self-attention refinement via one-step belief propagation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.
- Li, D., Wang, H., Shao, R., Guo, H., Xing, E., and Zhang, H. MPCFORMER: FAST, PERFORMANT AND PRIVATE TRANSFORMER INFERENCE WITH MPC. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023a.
- Li, H., Wang, M., Lu, S., Cui, X., and Chen, P.-Y. How do nonlinear transformers learn and generalize in in-context learning? In *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- Li, L. H., Hessel, J., Yu, Y., Ren, X., Chang, K.-W., and Choi, Y. Symbolic chain-of-thought distillation: Small models can also” think” step-by-step. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023b.
- Li, P., Yin, L., and Liu, S. Mix-LN: Unleashing the power of deeper layers by combining pre-LN and post-LN. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Li, Z., Ghodrati, S., Yazdanbakhsh, A., Esmailzadeh, H., and Kang, M. Accelerating attention through gradient-based learned runtime pruning. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022.
- Liang, C., Zuo, S., Zhang, Q., He, P., Chen, W., and Zhao, T. Less is more: Task-aware layer-wise distillation for language model compression. In *International Conference on Machine Learning (ICML)*, 2023.
- Lu, W.-j., Huang, Z., Gu, Z., Li, J., Liu, J., Ren, K., Hong, C., Wei, T., and Chen, W. Bumblebee: Secure two-party inference framework for large transformers. In *Annual Network and Distributed System Security Symposium (NDSS)*, 2025.

- Lu, X. and Van Roy, B. Information-theoretic confidence bounds for reinforcement learning. In *Advances in Neural Information Processing Systems*, 2019.
- Lu, X., Zhao, Y., and Qin, B. How does architecture influence the base capabilities of pre-trained language models? a case study based on ffn-wider transformer models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Ma, W., Zhang, K., Lou, R., Wang, L., and Vosoughi, S. Contributions of transformer attention heads in multi- and cross-lingual tasks. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
- Michel, P., Levy, O., and Neubig, G. Are sixteen heads really better than one? In *Advances in neural information processing systems*, 2019.
- Miller, D., Rao, A. V., Rose, K., and Gersho, A. A global optimization technique for statistical classifier design. *IEEE transactions on signal processing*, 1996.
- Mireshghallah, N., Kim, H., Zhou, X., Tsvetkov, Y., Sap, M., Shokri, R., and Choi, Y. Can LLMs keep a secret? testing privacy implications of language models via contextual integrity theory. In *The Twelfth International Conference on Learning Representations*, 2024.
- Mirzadeh, S. I., Alizadeh-Vahid, K., Mehta, S., del Mundo, C. C., Tuzel, O., Samei, G., Rastegari, M., and Farajtabar, M. ReLU strikes back: Exploiting activation sparsity in large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Mnih, V. Asynchronous methods for deep reinforcement learning. In *Proceedings of The 33rd International Conference on Machine Learning (ICML)*, 2016.
- Moon, J., Yoo, D., Jiang, X., and Kim, M. Thor: Secure transformer inference with homomorphic encryption. In *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security (ACM CCS)*, 2025.
- Mu, J. and Andreas, J. Compositional explanations of neurons. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., et al. Olmoe: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.
- Nahshan, Y., Kampeas, J., and Haleva, E. Linear log-normal attention with unbiased concentration. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Nanda, N. Attribution patching: Activation patching at industrial scale. URL: <https://www.neelnanda.io/mechanistic-interpretability/attribution-patching>, 2023.
- Neu, G., Jonsson, A., and Gómez, V. A unified view of entropy-regularized markov decision processes. *arXiv preprint arXiv:1705.07798*, 2017.
- Ni, Y., Guo, Y., Jia, J., and Huang, L. On the nonlinearity of layer normalization. In *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- Noci, L., Anagnostidis, S., Biggio, L., Orvieto, A., Singh, S. P., and Lucchi, A. Signal propagation in transformers: Theoretical perspectives and the role of rank collapse. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Noci, L., Li, C., Li, M., He, B., Hofmann, T., Maddison, C. J., and Roy, D. The shaped transformer: Attention models in the infinite depth-and-width limit. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Pang, Q., Zhu, J., Möllering, H., Zheng, W., and Schneider, T. Bolt: Privacy-preserving, accurate and efficient inference for transformers. In *IEEE Symposium on Security and Privacy (SP)*, 2024.
- Pearce, M. T., Doots, T., and Rigg, A. Weight-based decomposition: A case for bilinear MLPs. In *ICML 2024 Workshop on Mechanistic Interpretability*, 2024.
- Peer, D., Keulen, B., Stabinger, S., Piater, J., and Rodriguez-sanchez, A. Improving the trainability of deep neural networks through layerwise batch-entropy regularization. In *Transactions on Machine Learning Research (TMLR)*, 2022.
- Pereyra, G., Tucker, G., Chorowski, J., Kaiser, Ł., and Hinton, G. Regularizing neural networks by penalizing confident output distributions. *arXiv preprint arXiv:1701.06548*, 2017.
- Priyanshu, A., Vijay, S., Kumar, A., Naidu, R., and Mireshghallah, F. Are chatbots ready for privacy-sensitive applications? an investigation into input regularization and prompt-induced sanitization. *arXiv preprint arXiv:2305.15008*, 2023.
- Rybakov, O., Chrzanowski, M., Dykas, P., Xue, J., and Lanir, B. Methods of improving llm training stability. *arXiv preprint arXiv:2410.16682*, 2024.

- Salimans, T. and Kingma, D. P. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in neural information processing systems*, 2016.
- Setlur, A., Eysenbach, B., Smith, V., and Levine, S. Maximizing entropy on adversarial examples can improve generalization. In *ICLR 2022 Workshop on PAIR²Struct: Privacy, Accountability, Interpretability, Robustness, Reasoning on Structured Data*, 2022.
- Shannon, C. E. A mathematical theory of communication. *The Bell system technical journal*, 1948.
- Staab, R., Vero, M., Balunovic, M., and Vechev, M. Beyond memorization: Violating privacy via inference with large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Stanić, A., Ashley, D., Serikov, O., Kirsch, L., Faccio, F., Schmidhuber, J., Hofmann, T., and Schlag, I. The languini kitchen: Enabling language modelling research at different scales of compute. *arXiv preprint arXiv:2309.11197*, 2023.
- Veličković, P., Perivolaropoulos, C., Barbero, F., and Pascanu, R. Softmax is not enough (for sharp size generalisation). In *Forty-second International Conference on Machine Learning (ICML)*, 2025.
- Vig, J. and Belinkov, Y. Analyzing the structure of attention in a transformer language model. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, 2019.
- Voita, E., Talbot, D., Moiseev, F., Sennrich, R., and Titov, I. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.
- Wang, Y., Wang, L., Jiang, Y., Zou, W., Liu, T., Song, X., Wang, W., Xiao, L., Wu, J., Duan, J., et al. Diffusion actor-critic with entropy regulator. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Wei, X., Zhang, Y., Zhang, X., Gong, R., Zhang, S., Zhang, Q., Yu, F., and Liu, X. Outlier suppression: Pushing the limit of low-bit transformer language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Wortsman, M., Liu, P. J., Xiao, L., Everett, K. E., Alemi, A. A., Adlam, B., Co-Reyes, J. D., Gur, I., Kumar, A., Novak, R., Pennington, J., Sohl-Dickstein, J., Xu, K., Lee, J., Gilmer, J., and Kornblith, S. Small-scale proxies for large-scale transformer training instabilities. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Wu, X., Li, C., Aminabadi, R. Y., Yao, Z., and He, Y. Understanding int4 quantization for language models: latency speedup, composability, and failure cases. In *International Conference on Machine Learning (ICML)*, 2023.
- Wu, X., Ajorlou, A., Wang, Y., Jegelka, S., and Jadbabaie, A. On the role of attention masks and layernorm in transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations (ICML)*, 2024.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, 2020.
- Xu, T., Lu, W.-j., Yu, J., Chen, Y., Lin, C., Wang, R., and Li, M. Breaking the layer barrier: Remodeling private transformer inference with hybrid {CKKS} and {MPC}. In *34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- Yang, K., Weng, C., Lan, X., Zhang, J., and Wang, X. Ferret: Fast extension for correlated ot with small communication. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020.
- Ye, T., Dong, L., Xia, Y., Sun, Y., Zhu, Y., Huang, G., and Wei, F. Differential transformer. In *arXiv preprint arXiv:2410.05258*, 2024.
- Yin, Q., He, X., Zhuang, X., Zhao, Y., Yao, J., Shen, X., and Zhang, Q. Stablemask: Refining causal masking in decoder-only transformer. In *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- Yu, Z., Wang, Z., Fu, Y., Shi, H., Shaikh, K., and Lin, Y. C. Unveiling and harnessing hidden attention sinks: Enhancing large language models without training through attention calibration. In *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- Zeng, W., Li, M., Xiong, W., Lu, W., Tan, J., Wang, R., and Huang, R. Mpcvit: Searching for mpc-friendly vision transformer with heterogeneous attention. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

- Zhai, S., Likhomanenko, T., Littwin, E., Busbridge, D., Ramapuram, J., Zhang, Y., Gu, J., and Susskind, J. M. Stabilizing transformer training by preventing attention entropy collapse. In *International Conference on Machine Learning (ICML)*, 2023.
- Zhang, J., Liu, J., Yang, X., Wang, Y., Chen, K., Hou, X., Ren, K., and Yang, X. Secure transformer inference made non-interactive. In *Annual Network and Distributed System Security Symposium (NDSS)*, 2025a.
- Zhang, M., Bhatia, K., Kumbong, H., and Re, C. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- Zhang, Y., Chen, D., Kundu, S., Li, C., and Beerel, P. A. Sal-vit: Towards latency efficient private inference on vit using selective attention search with a learnable softmax approximation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Zhang, Z., Wang, Y., Huang, X., Fang, T., Zhang, H., Deng, C., Li, S., and Yu, D. Attention entropy is a key factor: An analysis of parallel context encoding with full-attention-based pre-trained language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025b.
- Zhao, B., Tu, H., Wei, C., Mei, J., and Xie, C. Tuning LayerNorm in attention: Towards efficient multi-modal llm finetuning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Zhao, S., Gong, M., Liu, T., Fu, H., and Tao, D. Domain generalization via entropy regularization. In *Advances in neural information processing systems (NeurIPS)*, 2020.
- Zhu, J., Chen, X., He, K., LeCun, Y., and Liu, Z. Transformers without normalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zimmerman, I., Baruch, M., Drucker, N., Ezov, G., Soceanu, O., and Wolf, L. Converting transformers to polynomial form for secure inference over homomorphic encryption. In *International Conference on Machine Learning (ICML)*, 2024.

A EXTENDED RELATED WORK

The Pitfalls of LayerNorm in LLM While LayerNorm serves as a critical source of nonlinearity in both convolutional neural networks (Ni et al., 2024) and transformer-based models (Wu et al., 2024; Zhao et al., 2024; Joudaki et al., 2023), it poses several challenges in the development and deployment of LLMs across various applications. In private LLMs, LayerNorm poses challenges due to the difficulty of inverse-square root computations and the complexity of polynomial approximations caused by its unusually wide range of variance values (Zimerman et al., 2024).

Furthermore, LayerNorm’s trainable parameters are strongly linked to outlier features (He et al., 2024; Bondarenko et al., 2023; Wei et al., 2022; Kovaleva et al., 2021), posing significant challenges for low-precision training and quantization. In particular, the LayerNorm scaling parameters amplify outlier activations, making it difficult to train models with fewer bits of precision (Wei et al., 2022).

LayerNorm also complicates mechanistic interpretability by making the residual stream more intricate and harder to analyze (Nanda, 2023). Finally, from a signal propagation perspective, LayerNorm can degrade trainability in deeper LLMs (He & Hofmann, 2024; He et al., 2023).

Entropy Regularization Entropy regularization has been widely applied in various areas of machine learning, serving multiple purposes: improving model robustness through penalizing low entropy predictions (Pereyra et al., 2017) or maximizing prediction entropy (Setlur et al., 2022), improving adversarial robustness (Jagatap et al., 2022), avoiding poor initialization and local minima (Miller et al., 1996), and optimizing layer-wise information flow (Peer et al., 2022).

In the context of attention mechanism, entropy regularization specifically targets attention score distributions. (Bao et al., 2024) leverage a signal propagation framework to regulate attention weight localization by penalizing the QK-eigenspectrum scale, thereby preventing signal vanishing and ensuring stable training dynamics.

In reinforcement learning, entropy-regularization is used for balancing exploration-exploitation trade-offs and promoting action diversity (Wang et al., 2024; Ahmed et al., 2019; Lu & Van Roy, 2019; Neu et al., 2017; Mnih, 2016). Also, it is used in domain generalization (Zhao et al., 2020).

B INTEGRATIONS OF ENTROPY REGULARIZATION IN LOSS FUNCTION

B.1 Entropy Regularization Algorithm

We present our entropy regularization in Algorithm 1, incorporating (i) head-wise learnable threshold for adaptable regularization, and 2) tolerance margin to prevent excessive

regularization, allowing a handful attention head to possess higher attention values.

B.2 PyTorch Implementation

The PyTorch implementation, shown in Figure 13, computes the entropy regularization loss for attention weights in a transformer model. This regularization ensures a balanced attention distribution, preventing it from becoming overly concentrated or too diffuse.

C WEIGHT AND SPECTRAL NORMALIZATION

Weight normalization reparameterizes the weight vectors as $\mathbf{W}_{\text{normalized}} = \frac{\mathbf{V}}{\|\mathbf{V}\|_2} g$, where $\mathbf{V}$ is reparameterized weight vector, $\|\mathbf{V}\|_2$ is Euclidean norm, and g is a learnable scaling factor. Spectral normalization, in contrast, normalizes the weight matrix $\mathbf{W}$ by its largest singular value $\sigma(\mathbf{W})$, yielding $\mathbf{W}_{\text{normalized}} = \frac{\mathbf{W}}{\sigma(\mathbf{W})}$. While weight normalization regulates weight magnitude, spectral normalization constrains the Lipschitz constant of linear layers. We employed these normalizations in the FFN of the softmax-only model which transform $\text{FFN}^{\text{SM}}(\mathbf{X}) = (\mathbf{X}\mathbf{W}_{\text{in}}^{\text{ffn}})\mathbf{W}_{\text{out}}^{\text{ffn}}$ as follows:

$$\begin{aligned} \text{FFN}_{\text{WNorm}}^{\text{SM}}(\mathbf{X}) &= \left(\mathbf{X} \frac{\mathbf{V}_{\text{in}}^{\text{ffn}}}{\|\mathbf{V}_{\text{in}}\|_2} g_{\text{in}} \right) \frac{\mathbf{V}_{\text{out}}^{\text{ffn}}}{\|\mathbf{V}_{\text{out}}\|_2} g_{\text{out}} \text{ and} \\ \text{FFN}_{\text{SNorm}}^{\text{SM}}(\mathbf{X}) &= \left(\mathbf{X} \frac{\mathbf{W}_{\text{in}}^{\text{ffn}}}{\sigma(\mathbf{W}_{\text{in}}^{\text{ffn}})} \right) \frac{\mathbf{W}_{\text{out}}^{\text{ffn}}}{\sigma(\mathbf{W}_{\text{out}}^{\text{ffn}})} \end{aligned} \quad (7)$$

Table 9 compares the performance of weight and spectral normalization applied to the linear layers within the attention and FFN sub-blocks in Softmax-only model. Results show that applying these normalization techniques to the attention blocks yields diminishing returns compared to their application in the FFN linear layers.

Table 9. Comparison of weight normalization (Salimans & Kingma, 2016) and spectral normalization (Miyato et al., 2018) when employed in Softmax-only GPT-2 ($L=12$, $H=12$, $d=768$) models, and trained from scratch ($T=128$) on CodeParrot dataset.

Linear layers	Eval PPL(WNorm.)	Eval PPL(SNorm.)
QK	3.89	4.25
FFN	3.64	3.63
QK+FFN	3.88	4.23
QKV+FFN	3.93	4.26
QKVO+FFN	3.98	4.34

D ENTROPY DYNAMICS

Figure 12 presents the entropy dynamics of the GPT-2 ($L=12$, $H=12$, $d=3072$) models as nonlinearities are progressively removed. Each model is trained from scratch on 2.1B tokens from CodeParrot dataset.

Algorithm 1 Entropy Regularization Loss Computation

Inputs: attentions: List of attention matrices, $\Theta(L, H)$ = reg_threshold_weights, T : Sequence length, λ : Regularization loss weightage, γ : Hyper-parameter for Tolerance margin

Output: $\mathcal{L}_{\text{total}}$: Total loss including entropy regularization

```

1:  $\mathcal{L}_{\text{entropy}} \leftarrow 0$ 
2:  $E_{\text{max}} \leftarrow \log(T)$  // Theoretical maximum value of entropy
3:  $\text{Tol}_{\text{margin}} \leftarrow \gamma E_{\text{max}}$  // Tolerance margin is set as a small fraction of  $E_{\text{max}}$ 
4: for each layer  $l$  in layers do
5:    $\mathcal{L}_{\text{layer}} \leftarrow 0$ 
6:    $A(t) \leftarrow \text{attentions}[l]$  // Attention matrix with learnable temperature for each query position
7:    $E(t) \leftarrow -\frac{1}{T} \sum_{i=1}^T \sum_{j=1}^T A_{ij}(t) \log(A_{ij}(t))$  // Compute entropy, averaged over query length
8:   for each head  $h$  in heads do
9:      $E^{(l,h)} \leftarrow \text{Slice}(E(t), h)$  // Entropy for head  $h$ 
10:     $\theta^{(l,h)} \leftarrow \text{Slice}(\Theta(L, H), h)$  // Learnable threshold weight head  $h$ 
11:     $\delta^{(l,h)} \leftarrow E^{(l,h)}(t) - \theta^{(l,h)} E_{\text{max}}$  // Deviation from head-specific threshold
12:     $\text{penalty}^{(l,h)} \leftarrow (\delta^{(l,h)})^2 \mathbb{1}(|\delta^{(l,h)}| > \text{Tol}_{\text{margin}})$  // Penalize iff deviation exceeds Tolerance
13:     $\mathcal{L}_{\text{layer}} \leftarrow \mathcal{L}_{\text{layer}} + \text{penalty}^{(l,h)}$ 
14:   end for
15:    $\mathcal{L}_{\text{layer}} \leftarrow \frac{\mathcal{L}_{\text{layer}}}{\text{num\_heads}}$  // Average over heads
16:    $\mathcal{L}_{\text{entropy}} \leftarrow \mathcal{L}_{\text{entropy}} + \mathcal{L}_{\text{layer}}$ 
17: end for
18:  $\mathcal{L}_{\text{entropy}} \leftarrow \frac{\mathcal{L}_{\text{entropy}}}{\text{len}(\text{attentions})}$  // Average over layers
19:  $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{entropy}}$ 
20: return  $\mathcal{L}_{\text{total}}$ 

```

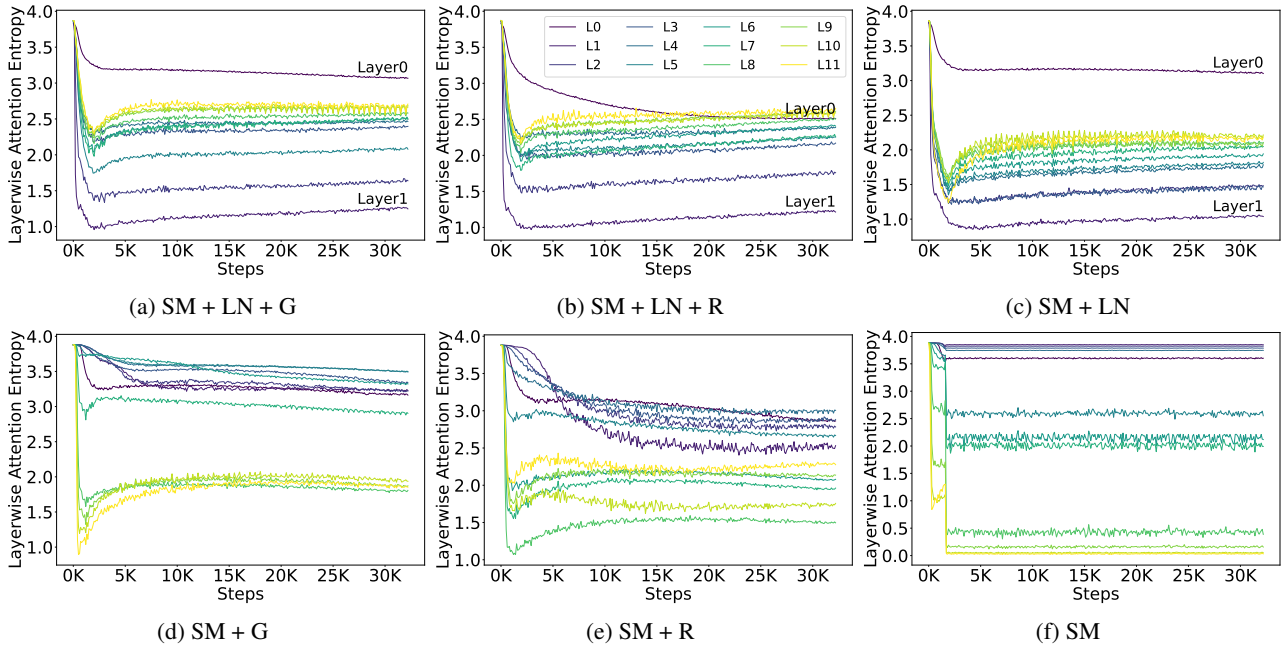


Figure 12. Entropy dynamics of GPT-2 (12L, 12H, 768d) models, each trained from scratch on 2.1B CodeParrot tokens with progressively fewer nonlinearities to isolate their effects. Normalization-free GELU (SM+G) and Softmax-only (SM) models show entropic overload in early layers; the latter also exhibits entropy collapse in deeper layers

```
1 import torch

3 def calculate_entropy_reg_loss(attentions, blocks, seq_len):
4     """
5     Calculate the entropy regularization loss.

7     Parameters:
8     attentions (list): A list of attention matrices from different layers.
9     blocks (list): A list of transformer blocks.
10    seq_len (int): The length of the sequence (context length).

12    Returns:
13    float: The entropy regularization loss.
14    """
15    entropy_reg_loss = 0
16    max_entropy = torch.log(torch.tensor(seq_len)) # Theoretical maximum entropy
17    fraction = 0.10 # Design hyper-parameter for tolerance margin
18    tolerance_margin = fraction * max_entropy # Set tolerance margin as fraction of
        the maximum entropy

20    for layer_idx, (block, attn_mat) in enumerate(zip(blocks, attentions)):
21        reg_threshold_weights = block.attn.reg_threshold_weights # Head-wise
            learnable parameters to set head-specific threshold
22        ent_val = -torch.sum(attn_mat * torch.log(attn_mat + 1e-9), dim=-1) # Compute
            entropy averaged over sequence length
23        layer_entropy_reg_loss = 0

25        for head_idx in range(block.attn.num_heads):
26            head_entropy = ent_val[:, head_idx, :] # Get head-specific entropy
27            threshold = reg_threshold_weights[head_idx] * max_entropy
28            deviation = torch.abs(head_entropy - threshold)
29            penalty = torch.square(torch.where(deviation > tolerance_margin, deviation
            , torch.zeros_like(deviation)))
30            layer_entropy_reg_loss += penalty.sum()

32        layer_entropy_reg_loss /= block.attn.num_heads
33        entropy_reg_loss += layer_entropy_reg_loss

35    entropy_reg_loss /= len(attentions)
36    return entropy_reg_loss

38 # Calculate the total loss including entropy regularization
39 lambda_reg = 1e-5 # Hyperparameter for entropy regularization weight
40 entropy_regularization = calculate_entropy_reg_loss(attentions, blocks, seq_len)
41 total_loss = ce_loss + lambda_reg * entropy_regularization
```

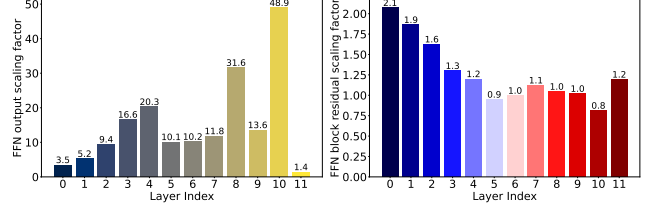
Figure 13. PyTorch implementation of entropy regularization loss calculation.

E LEARNABLE SCALING IN FEED-FORWARD NETWORKS

We plot the values of FFN scaling factors α and β (see Eq. 5) learned across the layers in the full-trained softmax-only GPT-2 model, and made the following observations from the Figure 14:

- **Significant increase in α with layer depth:** The scaling factor α increases substantially in the deeper layers of the model, with particularly high values observed in L_{10} (Figure 14a). This indicates that as the network goes deeper, the FFN outputs are heavily scaled down by α . This downscaling is essential to prevent the FFN outputs from dominating the activations, which could otherwise lead to numerical instability, as evidenced by the NaNs observed early in training in Figure 7a. The large α values in deeper layers suggest that this downscaling becomes increasingly critical as the model progresses through its layers, effectively stabilizing the training process by keeping the FFN outputs in check.
- **Balancing β values:** The scaling factors β , which modulate the residual connections within the FFN block by upscaling their output, start higher in the earlier layers and decreases gradually, with some fluctuation (Figure 14b). The moderate up-scaling provided by β ensures that the residual connections are not overshadowed by the scaled-down FFN outputs. This balance between the strong downscaling by α and the corresponding up-scaling by β is crucial for stabilizing activations across layers, particularly in deeper layers where instability is more likely to occur.

These observations underscore the critical role that the learnable scaling factors α and β play in stabilizing the training of softmax-only GPT-2 models. By dynamically adjusting the contributions of the FFN sub-block outputs, α and β prevent the numerical issues that arise in deeper layers, ensuring stable and effective training. This fine-tuned balance is key to avoiding entropy collapse and other forms of instability that would otherwise derail the training process.



(a) FFN scaling factor α (b) FFN residual scaling factor β

Figure 14. Learned scaling factors α and β in Eq. 5 across different layers in the Softmax-only GPT-2 model ($L=12$, $H=12$, $d=768$). The values were plotted after full model training to observe the modulation of FFN outputs and residual connections in each layer.

F MITIGATING OVER-REGULARIZATION THROUGH TOLERANCE MARGIN

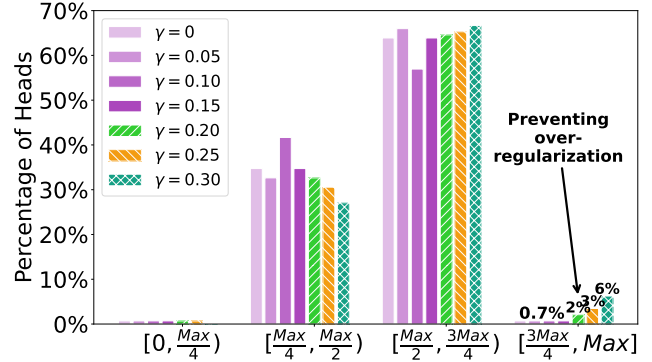


Figure 15. Headwise entropy distribution in Softmax-only GPT-2 model ($12L$, $12H$, $768d$) when entropy regularization is applied with varying threshold margin, controlled by hyperparameter γ .

Figure 15 illustrates the effect of γ on the headwise entropy distribution. The hyperparameter γ employed to adjust the threshold margin in entropy regularization, defined as $Tol_{margin} = \gamma E_{max}$ (Algorithm 1, line #1), effectively preventing over-regularization by ensuring that a sufficient fraction of heads maintains entropy values in the upper range $\frac{3Max}{4}$ to Max . As γ increases from 0 to 0.15, only a small proportion of attention heads (0.7%) are situated in the highest entropy range. However, as γ is increased beyond 0.15, the fraction of heads in this upper range starts increasing, reaching 2.08%, 3.47%, and 6.25% at $\gamma=0.20$, 0.25, and 0.30, respectively. This fine-grained control on the population of attention heads in the higher entropy range highlights the ability of entropy regularization to prevent over-regularization and maintain the attention heads’ diversity. We find that $\gamma=0.2$ yields slightly better performance in terms of lower perplexity compared to higher γ values, and thus, we adopt this value in the final entropy regularization scheme.

Furthermore, Figure 16 illustrates the layer-wise entropy dynamics during training as the tolerance margin is progressively increased. Evidently, at higher γ , a greater fraction of attention heads exhibit higher entropy values, as reflected in the increased mean entropy of the early layers. This prevents over-regularization while preserving the attention

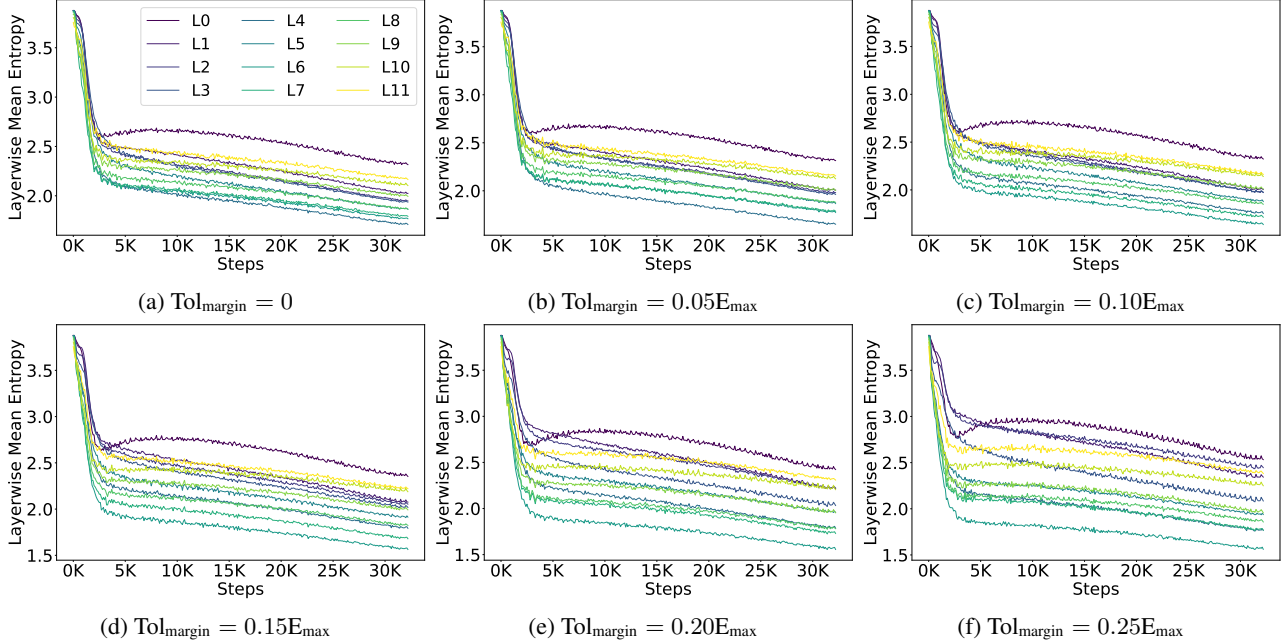


Figure 16. Layerwise entropy dynamics when entropy regularization is employed with increasing threshold margin, defined as $\text{Tol}_{\text{margin}} = \gamma E_{\text{max}}$ (see Algorithm 1, line #1). At higher γ , the mean entropy of the early layers increases.

head diversity.

G ADDITIONAL RESULTS

G.1 Baseline GPT-2 Models

Table 10 shows the detailed breakdown of private inference (communication and latency) overheads of baseline GPT-2 models. We compare two architectural variants for each baseline: the traditional GELU-based architecture (SM + LN + G) and the PI-friendly ReLU variant (SM + LN + R).

Table 10. Performance evaluation of baseline GPT-2 models trained from scratch on the CodeParrot dataset (2.1B tokens). L , H , d , and T represent model depth, head counts, embedding dimension, and input context length during training, respectively.

Config.	Network Arch.	Eval PPL	#Nonlinear Ops	Comm. (GBs)	Latency (minutes)
$L=18$ $H=12$ $d=768$ $T=128$	SM + LN + G	2.56	SM: $216 \times \mathbb{R}^{128 \times 128}$ LN: $36 \times \mathbb{R}^{128 \times 768}$ G: $18 \times \mathbb{R}^{128 \times 3072}$	37.17	10.77
	SM + LN + R	2.63	SM: $216 \times \mathbb{R}^{128 \times 128}$ LN: $36 \times \mathbb{R}^{128 \times 768}$ R: $18 \times \mathbb{R}^{128 \times 3072}$	13.34	8.04
$L=12$ $H=12$ $d=768$ $T=256$	SM + LN + G	2.35	SM: $144 \times \mathbb{R}^{256 \times 256}$ LN: $24 \times \mathbb{R}^{256 \times 768}$ G: $12 \times \mathbb{R}^{256 \times 3072}$	58.51	16.57
	SM + LN + R	2.41	SM: $144 \times \mathbb{R}^{256 \times 256}$ LN: $24 \times \mathbb{R}^{256 \times 768}$ R: $12 \times \mathbb{R}^{256 \times 3072}$	26.73	12.59

The ReLU-based design significantly improves PI efficiency, with a marginal increase in perplexity (PPL). Notably, communication overhead is reduced by up to $2.79\times$ (from

37.17GB to 13.34GB in the 18-layer model), while inference latency decreases by $\sim 1.3\times$ across all configurations.

H FUTURE WORK

To further reduce non-linear operations, off-the-shelf head pruning techniques (Voita et al., 2019; Michel et al., 2019; Jo & Myaeng, 2020; Ma et al., 2021; Li et al., 2022) can be applied on top of AERO. Another approach is to explore linear softmax operations. However, these linear softmax operations sometimes introduce additional normalization layers or complex activation functions in the FFN (Zhang et al., 2024), which could increase the PI overheads, counteracting the intended efficiency improvements.

Additionally, incorporating weight and activation quantization (Wu et al., 2023; Xiao et al., 2023; Dettmers & Zettlemoyer, 2023) could further enhance the efficiency of private inference in our architecture.

Orthogonally, performance improvement techniques such as knowledge distillation (KD) can be employed to complement these optimizations (Ko et al., 2024; Liang et al., 2023; Gu et al., 2023; Hsieh et al., 2023; Li et al., 2023b).

Looking ahead, scaling AERO to more complex and deeper LLMs can be achieved by strategically combining techniques such as weight normalization, spectral normalization, and FFN output scaling. These methods can be applied selectively, with different layers using different techniques—

for instance, employing spectral normalization in early layers and FFN output scaling in deeper layers. This tailored approach could lead to better stability and efficiency in larger models.

I AERO BEYOND PRIVATE INFERENCE: BROADER IMPACT AND PROMISES

While AERO is developed for enabling efficient private inference in transformer-based language models, its architectural innovations and insights have far-reaching implications beyond privacy-preserving computations. Its principled approach for architectural simplifications and techniques for maintaining model stability offer valuable insights into the broader field of language model design and optimization. Here, we discuss how AERO’s principles and entropy-guided solutions can impact both architectural design choices and practical deployment considerations.

Plaintext efficiency Our finding that ReLU naturally emerges as the preferred activation function in normalization-free architectures aligns well with plaintext efficiency goals. ReLU’s ability to induce sparsity in activations accelerates the plaintext inference by reducing the data traffic between CPU and GPU (Mirzadeh et al., 2024).

Low-precision training and quantization for resource-constrained applications The deployment of LLMs in resource-constrained environments is significantly hindered by outliers that complicate low-bitwidth quantization. These outliers emerge from two primary sources: softmax-based attention mechanism (Hu et al., 2024a) and LayerNorm layers (He et al., 2024; Wei et al., 2022; Kovaleva et al., 2021). LayerNorm-induced outliers are particularly *severe* as their scaling parameters amplify activation extremes, demanding higher bit-widths for accurate representation.

Our normalization-free design inherently addresses the major source of outliers by eliminating problematic scaling parameters. While empirical validation remains necessary, AERO’s architectural choices offer promising directions for enabling low-precision training and quantization, where effective outlier management is crucial.

Entropy-guided framework for tackling softmax-inherent challenges in attention mechanism The softmax function, fundamental to transformer-based attention mechanisms, inherently assigns non-zero probabilities to all tokens due to its normalized exponential structure. This leads to two primary issues inherent to softmax: disproportionate emphasis on specific tokens (known as attention sink) (Xiao et al., 2024; Cancedda, 2024; Gu et al., 2024); and non-zero scores for irrelevant tokens (known as attention noise). These challenges can result in undesirable effects such as hallucinations (Ye et al., 2024), outlier activations (Hu et al., 2024a), and rank collapse (Bao et al., 2024).

While prior work proposed various strategies to mitigate these issues (Yin et al., 2024; Yu et al., 2024; Bao et al., 2024), our entropy regularization offers a fine-grained control of attention entropy distribution. By penalizing extreme entropy values using headwise learnable threshold parameters, each attention head adaptively determine its optimal degree of focus, preventing over-diffusion while *preserving* the mathematical properties of softmax.

Insights into mechanistic interpretability and disentangling polysemantic neurons By stripping away complex nonlinearities, such as GELU and LayerNorm, AERO makes it easier to dissect how individual neurons, attention mechanisms, and linear FFNs (in Softmax-only architecture) contribute to the model’s internal behavior, facilitating a more *granular* understanding of LLM internal dynamics. In standard transformer architectures, nonlinearities like GELU induce complex feature interactions, often resulting in polysemantic neurons—neurons that encode overlapping or unrelated information (Gurnee et al., 2023; Mu & Andreas, 2020). By simplifying the FFN to a purely linear structure, AERO could minimize feature entanglement, fosters monosemantic neuron behavior, and produces a more disentangled and interpretable neuron space (Pearce et al., 2024).